



```
In [1]: import numpy as np
```

```
In [65]: #Converting a LIST in python to an ARRAY  
#NOTE : ARRAY always has same type of data elements.  
#Meaning if you have string and int, arr would convert all to int  
my_list = [1,2,3]  
my_list_2 = ['1',2,3]  
  
my_arr = np.array(my_list)  
my_arr_2 = np.array(my_list_2)  
  
my_arr_2
```

```
Out[65]: array(['1', '2', '3'], dtype='<U11')
```

```
In [4]: #Creating matrices  
my_list = [  
    [1,2,3],  
    [4,5,6],  
    [7,8,9]  
]  
  
my_mat = np.array(my_list)  
my_mat
```

```
Out[4]: array([[1, 2, 3],  
              [4, 5, 6],  
              [7, 8, 9]])
```

```
In [10]: #Similar to for i in range BUT for arrays  
np.arange(-1,10,2)
```

```
Out[10]: array([-1,  1,  3,  5,  7,  9])
```

```
In [12]: np.zeros(3)  
#CREATE A zero matrix of size of parameter  
#NOTE : zeros would be in FLOAT
```

```
Out[12]: array([0., 0., 0.])
```

```
In [14]: np.zeros((5,5))
```

```
Out[14]: array([[0., 0., 0., 0., 0.],  
               [0., 0., 0., 0., 0.],  
               [0., 0., 0., 0., 0.],  
               [0., 0., 0., 0., 0.],  
               [0., 0., 0., 0., 0.]])
```

```
In [16]: #similarly we have 1s matrix  
#NOTE Again in float  
np.ones((5,5))
```

```
Out[16]: array([[1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.],
               [1., 1., 1., 1., 1.]])
```

```
In [67]: #Get STEP elements from LB to UB, EVENLY SPACED
         #np.linspace(LB,UB,STEP)
         np.linspace(0,2,10)
```

```
Out[67]: array([0.          , 0.22222222, 0.44444444, 0.66666667, 0.88888889,
               1.11111111, 1.33333333, 1.55555556, 1.77777778, 2.          ])
```

```
In [66]: #GETTING an IDENTITY matrix of any order
         np.eye(4)
```

```
Out[66]: array([[1., 0., 0., 0.],
               [0., 1., 0., 0.],
               [0., 0., 1., 0.],
               [0., 0., 0., 1.]])
```

```
In [22]: #This is similar to Random module in python
         #BUT it retrns in an ARRAY
         np.random.rand(5)
```

```
Out[22]: array([0.15103903, 0.15694191, 0.04821904, 0.75311641, 0.41386649])
```

```
In [24]: #Return values from a standard normal distribution in an ARRAY
         np.random.randn(3)
```

```
Out[24]: array([-1.85246471, -0.45339385, -0.31871318])
```

```
In [26]: np.random.randint(1,100,10)
```

```
Out[26]: array([56, 67, 17, 78, 47, 32, 71, 8, 4, 87])
```

```
In [32]: arr = np.arange(26)#generate values in an array
         arr.reshape(13,2)#reshape those values into a matrix of the given shape
```

```
Out[32]: array([[ 0,  1],
               [ 2,  3],
               [ 4,  5],
               [ 6,  7],
               [ 8,  9],
               [10, 11],
               [12, 13],
               [14, 15],
               [16, 17],
               [18, 19],
               [20, 21],
               [22, 23],
               [24, 25]])
```

```
In [35]: #Note : Parentheses is only applied on functions, not attributes of the object
#Eg for arr here, shape would be written as is
print(arr.min())
print(arr.max())
print(arr.shape)
print(arr.dtype)
```

```
0
25
(26,)
int32
```

```
In [45]: arr = np.arange(0,11)
print(arr)
print(f"arr[8] -> {arr[8]}")
print(f"arr[1:5] -> {arr[1:5]}")
print(f"arr[0:5] -> {arr[0:5]}")
# arr[1:5]
```

```
[ 0  1  2  3  4  5  6  7  8  9 10]
arr[8] -> 8
arr[1:5] -> [1 2 3 4]
arr[0:5] -> [0 1 2 3 4]
```

```
In [47]: arr_copy = arr.copy()
#NOW, if we change arr_copy, original array would not be changed.
arr[0:5]=100
arr_copy[0:5]=101
print(arr)
print(arr_copy)
```

```
[100 100 100 100 100  5  6  7  8  9 10]
[101 101 101 101 101  5  6  7  8  9 10]
```

```
In [43]: )
```

```
Out[43]: array([100, 100, 100, 100, 100,  5,  6,  7,  8,  9, 10])
```

```
In [58]: arr_2d = np.array([
    [5,10,15],
    [20,25,30],
    [35,40,45]
])

#INDEXING IN MATRICES
print(f"Element 1,1 -> {arr_2d[0][0]}")
arr_2d[0,0]#this also gives the same output

#SLICING in matrices
print(f"arr_2d[:2,1:] -> {arr_2d[:2,1:]} \n") #Cols 0 TILL (Not including) 2,
print(f"arr_2d[:2] ->{arr_2d[:2]}")
```

```
Element 1,1 -> 5
arr_2d[:2,1:] -> [[10 15]
 [25 30]]

arr_2d[:2] -> [[ 5 10 15]
 [20 25 30]]
```

```
In [63]: arr = np.arange(1,11)

bool_arr = arr>5

bool_arr#CREATES A FILTER THAT CAN BE APPLIED on the OG array

arr[bool_arr]# NEW FILTERED ARRAY

arr[arr>5]#SAME RESULT
```

```
Out[63]: array([ 6,  7,  8,  9, 10])
```

```
In [75]: arr = np.arange(0,10)

# print(arr+arr)
# print(arr-arr)
# print(arr*arr)
# print(arr/arr) #SHOWS runtime Warning not error, if your array contains 0
# print(1/arr) #same
# print(arr**3)

print(f"np.sqrt(arr) -> {np.sqrt(arr)}\n\n")
print(f"np.exp(arr) -> {np.exp(arr)}\n\n")
print(f"np.max(arr) -> {np.max(arr)}\n\n")
print(f"np.min(arr) -> {np.min(arr)}\n\n")
print(f"np.log(arr) -> {np.log(arr)}\n\n")
print(f"np.sin(arr) -> {np.sin(arr)}\n\n")
```

```
np.sqrt(arr) -> [0.          1.          1.41421356 1.73205081 2.          2.23606798
 2.44948974 2.64575131 2.82842712 3.          ]
```

```
np.exp(arr) -> [1.00000000e+00 2.71828183e+00 7.38905610e+00 2.00855369e+01
 5.45981500e+01 1.48413159e+02 4.03428793e+02 1.09663316e+03
 2.98095799e+03 8.10308393e+03]
```

```
np.max(arr) -> 9
```

```
np.min(arr) -> 0
```

```
np.log(arr) -> [      -inf 0.          0.69314718 1.09861229 1.38629436 1.60943791
 1.79175947 1.94591015 2.07944154 2.19722458]
```

```
np.sin(arr) -> [ 0.          0.84147098  0.90929743  0.14112001 -0.7568025
-0.95892427
-0.2794155   0.6569866   0.98935825  0.41211849]
```

```
C:\Users\student\AppData\Local\Temp\ipykernel_16860\2901067451.py:15: RuntimeWarning: divide by zero encountered in log
print(f"np.log(arr) -> {np.log(arr)}\n\n")
```

In [88]:

```
"""
Create a 1D array having 10 elements, all 1
"""
np.ones(10)
"""
Create a 1D array having 10 elements, all 7
"""
np.ones(10)*7

"""
Create a 1D array having even integers, from 10 to 50
"""
np.arange(10,51,2)

"""
Create a 3X3 array from 0 to 8
"""
np.arange(0,9).reshape(3,3)

#Create a matrix from 0.1 to 10 in a 10X10 shape with 0.1 step
np.arange(1,101).reshape(10,10)/10

#linearly spaced points 20, between 0 and 1
```

```
np.linspace(0,1,20).reshape(5,4)
```

```
Out[88]: array([[0.          , 0.05263158, 0.10526316, 0.15789474],  
               [0.21052632, 0.26315789, 0.31578947, 0.36842105],  
               [0.42105263, 0.47368421, 0.52631579, 0.57894737],  
               [0.63157895, 0.68421053, 0.73684211, 0.78947368],  
               [0.84210526, 0.89473684, 0.94736842, 1.          ]])
```